```
import { q, one } from '../db.js';

// The capability registry. A capability is a named verb with a handler.
// Nothing in the platform executes except through here.
const handlers = new Map();

export function defineCapability(def) {
  if (!def.key || typeof def.handler !== 'function') {
    throw new Error('capability needs key and handler');
  }
  handlers.set(def.key, def);
  return def;
}

export function getCapability(key) {
  return handlers.get(key) ?? null;
}

export function listCapabilities() {
  return [...handlers.values()].map(({ handler, verify, plan, ...rest }) => rest)
}

export async function persistCapabilities() {
  for (const c of handlers.values()) {
    await q(
      `insert into capability (key, package_key, summary, input_schema, output_sc
       values ($1,$2,$3,$4,$5,$6)
       on conflict (key) do update set summary = excluded.summary,
         input_schema = excluded.input_schema, sensitivity = excluded.sensitivity
      [c.key, c.packageKey ?? 'kernel', c.summary ?? null,
       JSON.stringify(c.input ?? {}), JSON.stringify(c.output ?? {}),
       c.sensitivity ?? 'normal']
    );
  }
}

// Grants beat roles. A membership-specific grant wins over a role grant.
// Absence of any grant is a denial, not a default-allow.
export async function resolveDisposition({ businessId, membership, capability, re
  if (!membership) return 'never';

  // The platform acting on the business's behalf (fan-out, scheduled work).
  // An explicit denial still stops it; absence of a grant does not.
  if (system) {
```

```
    const denied = await q(
      `select 1 from grant_row where business_id = $1 and capability = $2 and dis
      [businessId, capability]
    );
    return denied.length ? 'never' : 'auto';
  }

  const rows = await q(
    `select disposition, membership_id, role, resource
       from grant_row
      where business_id = $1
        and capability = $2
        and (membership_id = $3 or role = $4)`,
    [businessId, capability, membership.id, membership.role]
  );

  const applicable = rows.filter(r => r.resource === '*' || r.resource === resour
  if (applicable.length === 0) return 'never';

  // Explicit denial anywhere wins.
  if (applicable.some(r => r.disposition === 'never')) return 'never';
  // Membership-specific grant beats role grant.
  const specific = applicable.find(r => r.membership_id === membership.id);
  if (specific) return specific.disposition;
  // Most permissive remaining role grant.
  return applicable.some(r => r.disposition === 'auto') ? 'auto' : 'ask';
}

export async function grant({ businessId, membershipId = null, role = null, capab
  return one(
    `insert into grant_row (business_id, membership_id, role, capability, resourc
     values ($1,$2,$3,$4,$5,$6) returning *`,
    [businessId, membershipId, role, capability, resource, disposition]
  );
}

export async function revokeForInstall({ businessId, packageKey }) {
  await q(
    `delete from grant_row where business_id = $1 and capability like $2`,
    [businessId, packageKey + '.%']
  );
}
```